RAG Chatbot -- Project Overview

Retrieval-Augmented Generation chatbot with citations, swappable providers, and a Streamlit UI.

What it is

A production-quality Retrieval-Augmented Generation (RAG) chatbot. You upload PDF / TXT / Markdown / DOCX documents into a local corpus, ask questions, and receive answers that are grounded in -- and cite back to -- the exact chunks of those documents. No hallucinated facts; if the answer is not in the corpus, the bot says so plainly.

What it does (end-to-end flow)

1. Ingest: load a document, split into ~800-char chunks with overlap, embed each chunk with a sentence-transformer model, store in ChromaDB with deterministic chunk ids (re-uploads dedupe automatically).
2. Retrieve: embed the user's question, run cosine search against the vector store (optionally fuse with BM25 keyword scores via Reciprocal Rank Fusion). Return the top-k relevant chunks.
3. Generate: build a prompt that places the retrieved chunks in a <context> block alongside the question, stream the LLM's reply token by token over Server-Sent Events.
4. Cite: every retrieved chunk that was shown to the model is surfaced as a citation (source file, page, score, snippet) so the user can verify the answer.

Stack

API: FastAPI + uvicorn (Python 3.11+)
LLM: Anthropic Claude (claude-opus-4-7) or Google Gemini (gemini-2.5-flash) -- swappable in one file.
Embeddings: sentence-transformers, default BAAI/bge-small-en-v1.5.
Vector DB: ChromaDB (persistent, local, cosine similarity).
Chunking: Recursive character splitter with deterministic SHA-256 ids.
UI: Streamlit (sidebar uploader + chat panel + sources panel).
Config: pydantic-settings reading .env.
Logging: structlog.

Key features

- Streaming responses (SSE) with live token deltas in the UI.
- Inline citations + a separate Sources panel showing scores and chunk snippets.
- Session memory: follow-up questions are query-rewritten into standalone search queries using prior turns.
- Dedupe-on-reingest: re-uploading the same file produces a net-zero change to the vector store.
- Pluggable interfaces (LLM, Embedder, VectorStore are ABCs) -- switching providers is a one-file change.
- Strict quality bar: mypy --strict clean, ruff clean, ~156 tests covering ingestion, retrieval, generation, and the HTTP API.

Why use RAG instead of a plain LLM?

Plain LLMs hallucinate when asked about content they were not trained on. RAG grounds answers in a private document corpus you control, so the model can answer questions about *your* docs (policies, manuals, research papers, codebases) with verifiable citations. The system prompt is deliberately strict: the model is told to refuse rather than guess.

How to run (one-screen summary)

1. uv sync --extra dev
2. Copy .env.example to .env and set GEMINI_API_KEY (or ANTHROPIC_API_KEY) + LLM_PROVIDER.
3. Terminal 1: uv run uvicorn app.main:app --reload
4. Terminal 2: uv run streamlit run ui/streamlit_app.py

RAG Chatbot -- Project Overview

Retrieval-Augmented Generation chatbot with citations, swappable providers, and a Streamlit UI.

5. Open http://localhost:8501, upload a document via the sidebar, ask questions.

See QUICKSTART.md and README.md for the full guide.

Non-goals (v1)

Authentication, multi-tenant isolation, fine-tuning, agentic tool use beyond retrieval, and Kubernetes deployment configs are deliberately out of scope. Docker packaging is fine to add; orchestration is not.